

Authentication in Healthcare Systems

Giuseppe Pisano
g.pisano@unibo.it

Healthcare Data Privacy and Security

August, 2026



S.K.I.L.L.E.D — AID013244/04/2

Strategic Knowledge and Inclusive Lifelong Learning in the health sector for youth Employability and Development

- 1 Authentication Foundations
- 2 Authentication Factors
- 3 Passwords and Password Files
- 4 Password Policy
- 5 Tokens and Smart Cards
- 6 Remote Authentication
- 7 Attacks Against Authentication

8 Multi-Factor and One-Time Passwords

9 Authentication Lab

10 Connecting the Concepts

Throughout the lesson we follow **St. Isidore Hospital**, a mid-size hospital with an electronic health record, a laboratory system, a pharmacy module, a patient portal, shared ward workstations, and remote access for clinicians on call.

The authentication question

Before the hospital can decide what a person may do, it must decide whether the person is really the doctor, nurse, pharmacist, patient, or administrator they claim to be.

- 1 We start with the meaning of **authentication**: identity claims, identifiers, and verification.
- 2 We classify authentication factors: *knowledge*, *possession*, *inherence*, and *behavior*.
- 3 We study passwords as the most common mechanism, including hashes, salts, cracking, and policy.
- 4 We move to physical and logical tokens, smart cards, and remote challenge-response.
- 5 We finish with attacks, multi-factor authentication, one-time passwords, and modern phishing-resistant authenticators.

Authentication connects several parts of the security architecture.

Relationship with earlier topics

Access control asks what an authenticated subject is allowed to do. Authentication asks whether the subject is really who they claim to be.

Relationship with later topics

Encryption protects credentials and sessions in transit and at rest. **GDPR** requires appropriate security for personal data, which includes preventing unauthorized access to health records.

Authentication Foundations

Section Context: Authentication Foundations

What we are talking about

Authentication is the security function that connects an **identity claim** to evidence. It comes before access control because the system cannot apply a policy to “Dr. Moretti” unless it first verifies that the actor really is Dr. Moretti.

Hospital lens

At St. Isidore Hospital, the same foundation applies to staff logins, patient portal accounts, medical devices, and service accounts used by applications.

Why Authentication Matters

Authentication is the process that verifies a claimed identity of, or for, a system [Shirey, 2007]. In practice, it is the first line of defense because every later control depends on it.

Hospital example

If someone opens the oncology record of patient Maria Rossi, the hospital needs more than “a workstation did it.” It needs to know whether the actor was Dr. Moretti, a ward nurse, a lab technician, or an attacker using an abandoned session.

Core idea

Authentication creates accountability: actions can be linked to a verified identity, not merely to a device or network address.

Identification and Verification

Authentication usually has two steps.

Identification

The subject presents an **identifier**: a username, badge ID, employee number, device name, API client ID, or patient portal account.

Verification

The subject presents or generates **authentication information** that proves the identifier belongs to them: a password, PIN, token response, biometric sample, certificate signature, or OTP.

Hospital example

The login name `l.bianchi` identifies a nurse. Her password and second factor verify that the person typing is likely Nurse Bianchi.

Authentication Is Not Authorization

These terms are related, but they answer different questions.

Three questions

Identification: Who are you claiming to be?

Authentication: Can you prove that claim?

Authorization: Given that identity, what are you allowed to do?

Hospital example

Dr. Moretti can authenticate successfully and still be denied access to payroll files. Authentication proves identity; it does not automatically grant every permission.

User and Message Authentication

The word authentication appears in more than one context.

User authentication

The system validates the identity of a human or service account before allowing interaction.

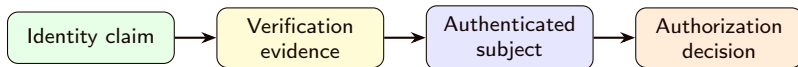
Message authentication

Communicating parties verify that a message was not altered and that it came from an authentic source.

Hospital example

A doctor logging into the EHR needs user authentication. A lab result sent from the laboratory system to the EHR needs message authentication so the EHR can trust both the source and the content.

Authentication Pipeline



Hospital example

`m.rossi` plus valid authentication evidence becomes the authenticated subject “Dr. Rossi.” The access-control system can then evaluate whether Dr. Rossi may prescribe a controlled drug.

Recap: Authentication Foundations

What to keep

Authentication binds a claimed identifier to evidence. It is different from authorization, but authorization depends on it. User authentication proves the actor; message authentication protects communication.

In St. Isidore Hospital

The hospital cannot safely enforce roles, audit accesses, or investigate incidents unless it can trust the identity attached to each action.

Authentication Factors

Section Context: Authentication Factors

What we are talking about

Authentication mechanisms differ by the kind of evidence they request. Classifying factors helps us understand why a password, a badge, a fingerprint, and a smart card do not fail in the same way.

Hospital lens

The hospital may use a password for the patient portal, a badge and PIN for medication cabinets, and stronger MFA for remote EHR access.

Four Factor Families

Authentication factors are commonly grouped into four families.

Knowledge

Something the user **knows**: password, PIN, recovery phrase, or answer to a secret question.

Possession

Something the user **has**: badge, smart card, hardware token, phone, cryptographic key, or authenticator app.

Inherence and behavior

Something the user **is**: face or fingerprint. Something the user **does**: voice pattern, handwriting dynamics, or typing rhythm.

Strengths and Weaknesses

No factor is perfect.

Knowledge factors

Passwords can be guessed, reused, phished, forgotten, or written down.

Possession factors

Tokens can be lost, stolen, duplicated, damaged, or unavailable during emergencies.

Biometrics

Biometrics can be convenient, but they raise false positive, false negative, privacy, cost, and recovery issues. A compromised password can be changed; a compromised fingerprint cannot be replaced in the same way.

Running Example: Same Login, Different Factors



Imagine three ways to access the medication administration record.

Password only

Nurse Bianchi types a username and password on a shared ward workstation.

Badge plus PIN

The workstation requires her hospital badge and a short PIN before opening the medication screen.

Smart card plus biometric unlock

For high-risk actions, such as approving chemotherapy, the system requires a smart card unlocked locally with a fingerprint.

Multi-Factor Authentication Principle

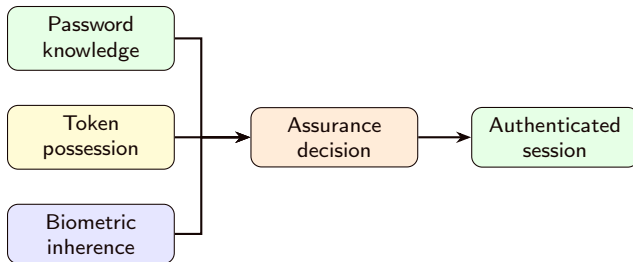
Definition

Multi-factor authentication combines evidence from **different factor families**. A password plus another password is still knowledge-only; a password plus a smart card is multi-factor.

Hospital example

Remote access to the EHR should not depend only on Dr. Moretti's password. A stolen password from a phishing email becomes less useful if the attacker also needs a hardware token or phishing-resistant authenticator.

Scheme: Combining Factors



Design point

More factors can increase assurance, but they also affect usability, recovery, support workload, and emergency access.

Recap: Authentication Factors

What to keep

Authentication factors are not interchangeable. Each factor family fails differently, so combining independent factors can reduce the chance that one compromise becomes full account access.

In St. Isidore Hospital

Password-only access may be acceptable for low-risk internal systems, but remote EHR access and high-impact clinical actions need stronger assurance.

Passwords and Password Files

Section Context: Passwords and Password Files

What we are talking about

Passwords are still the most common authentication mechanism. The security question is not only whether users choose good passwords, but also how the system stores, verifies, protects, and monitors password-derived data.

Hospital lens

If the EHR password database is stolen, the difference between plaintext passwords and salted slow hashes may decide whether the incident becomes a hospital-wide account compromise.

Password-Based Authentication

In a password system, the password verifies the user ID.

What the user ID does

It determines whether the account exists, which privileges are attached to it, and how the system refers to the user in access-control rules and logs.

What the password does

It provides evidence that the person using that user ID is the legitimate account holder.

Hospital example

The user ID `pharma.galli` links to pharmacy privileges. The password is supposed to prevent a different employee from using those privileges.

Password Files Store Verifiers

A well-designed system should not store plaintext passwords.

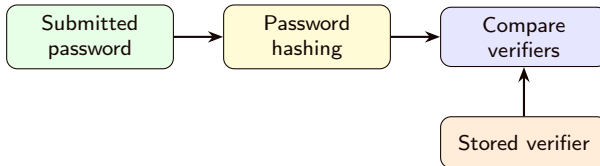
Stored verifier

The system stores a value derived from the password using a one-way password hashing function. During login, the system recomputes the verifier from the submitted password and compares it with the stored value.

Hospital example

If the EHR database is copied during an incident, plaintext passwords would immediately endanger every clinician account. Password hashes limit the damage, although weak passwords may still be cracked offline.

Scheme: Password Verification



Important

Hashing is not encryption. There should be no decryption key that recovers the original password.

Common Password Attacks

Attack	Hospital example
Offline dictionary attack	An attacker steals password hashes from an old backup and tests likely passwords on their own machine.
Specific account attack	A former contractor repeatedly tries to guess the head nurse's password.
Popular password attack	The attacker tries Password123!, season names, and hospital-themed passwords across many accounts.
Workstation hijacking	Someone uses a workstation where a clinician walked away without locking the session.
Password reuse	A password leaked from a personal site is reused for the patient portal.
Electronic monitoring	A keylogger captures credentials typed on a compromised endpoint.

Countermeasures Are Layered

Prevent guessing

Use minimum length, block known compromised passwords, avoid predictable composition rules, and rate-limit online attempts [National Institute of Standards and Technology, 2025].

Reduce value of stolen files

Protect password databases, use salted slow password hashes, restrict backups, monitor access, and force resets after confirmed compromise.

Protect active sessions

Use workstation locking, inactivity timeouts, reauthentication for sensitive actions, and strong session management.

Salted Password Hashes

A **salt** is a random value stored with a password verifier.

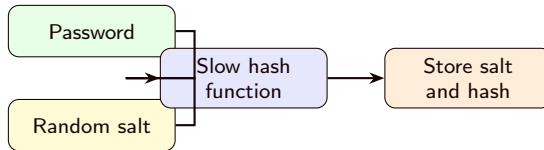
How it works

When a password is created, the system generates a unique salt, combines it with the password, and computes a password hash. The stored record contains the salt and the resulting verifier.

Hospital example

If two nurses choose the same password, their stored hashes should still be different. Otherwise, an attacker can immediately see password reuse inside the hospital.

Scheme: Salted Password Storage



Security effect

Salts make duplicate passwords less visible and make precomputed rainbow tables much less useful.

Why Salt Helps Against Cracking

Without salt

An attacker can compute a large table of password hashes once and reuse it against many systems.

With salt

The attacker must test candidate passwords together with each account's salt. The same password produces different stored values across users and systems.

Hospital example

A stolen radiology application backup contains salted hashes. The attacker can still try guesses, but cannot simply use a generic table of common password hashes.

Historical UNIX Password Schemes

Scheme	What it teaches
Original crypt(3)	DES-based, short password limit, small salt, repeated computation. Historically important, but obsolete for modern systems.
MD5 and SHA-based schemes	Longer passwords and more iterations, but MD5 is obsolete and simple fast hashes are not appropriate for password storage.
bcrypt	Uses a salt and a variable work factor so administrators can make cracking more expensive as hardware improves.

Modern note

Current systems should use password hashing functions designed for passwords, such as **bcrypt**, **scrypt**, **Argon2**, or platform-approved equivalents.

Password Cracking

Password cracking rarely tries every possible string first. It exploits human choices.

Dictionary and mutation attacks

Attackers start from common passwords, names, places, dates, keyboard patterns, and leaked password lists. Then they mutate them with uppercase letters, numbers, symbols, reversals, and substitutions.

Hospital example

StIsidore2026! looks complex, but it is built from public information and a year. A hospital-themed password is exactly the kind of candidate an attacker would try.

Rainbow Tables

Idea

A rainbow table trades storage for speed. Instead of hashing guesses during the attack, the attacker prepares large sets of password-hash relationships in advance.

Limits

Long salts and slow password hashing functions make generic precomputation much less useful. The attacker must do more account-specific work.

Hospital example

If an old departmental application stores unsalted MD5 password hashes, a public rainbow table may reveal many passwords quickly. If it stores salted bcrypt hashes, the attack becomes much more expensive.

Shadow Password Files

Shadow password files separate public account information from protected password verifiers.

Purpose

Normal users may need to read account names or user IDs, but they should not read password hashes. Shadow files restrict hash access to privileged components.

Residual risks

Operating-system vulnerabilities, permission mistakes, weak backup protection, password reuse, and network sniffing can still expose credentials or usable authentication data.

Recap: Passwords and Password Files

What to keep

Password systems depend on careful verifier storage, salts, slow hashing, rate limits, protected backups, and session controls. A hash leak is not harmless; it starts an offline race between attacker guesses and defensive design.

In St. Isidore Hospital

The hospital should assume some passwords will be weak or reused, so password security must include technical controls, monitoring, and MFA for sensitive systems.

Password Policy

Section Context: Password Policy

What we are talking about

Password policy shapes user behavior. A policy that looks strict on paper can be weak in practice if it encourages predictable choices, password reuse, or written notes.

Hospital lens

Clinical staff authenticate under time pressure. The policy must protect patients without making legitimate care workflows unnecessarily difficult.

The Policy Tension

Password policy must balance two goals.

Security goal

Eliminate passwords that are easy to guess, reuse, or find in breach lists.

Usability goal

Allow users to choose secrets they can remember without writing them on a monitor, sharing them with colleagues, or using predictable transformations.

Hospital example

A nurse moving between patients cannot safely manage a password rule that changes every month and requires obscure symbols. Bad usability becomes a security risk.

User Education and Passphrases

Education alone is not sufficient, but it helps users understand what the policy is trying to prevent.

Passphrase approach

A memorable sentence can be transformed into a longer secret. Length usually matters more than decorative complexity, provided the phrase is not obvious or reused.

Hospital example

A clinician might build a private passphrase from a personal sentence, not from the hospital name, ward name, favorite team, date of birth, or medical specialty.

Generated Passwords and Password Managers

Computer-generated passwords can be strong, but only if users have a usable way to store and use them.

Random generation

Random long passwords are difficult to guess but hard to memorize.

Password managers

A managed password vault can generate unique passwords for many systems, reducing reuse. In a hospital, this must be aligned with endpoint security, recovery procedures, and emergency access rules.

Reactive Password Checking

Method

The organization periodically runs a password cracker against its own password hashes to find weak passwords.

Problem

The weak password remains valid until the check finds it. The process also consumes resources and requires very careful handling of password hashes.

Hospital example

Security staff discover that several accounts use passwords based on ward names. Those users are forced to reset passwords, but the accounts were exposed before discovery.

Proactive Password Checking

Method

The system checks password quality at creation time and rejects weak choices before they become active.

Modern checks

Useful checks include minimum length, known-compromised password lists, blocklists of local terms, and detection of obvious patterns. Overly rigid composition rules can backfire.

Hospital example

The patient portal rejects StIsidore2026! because it contains the hospital name and a predictable year, even though it contains uppercase, lowercase, digits, and punctuation.

Lockout, Rate Limits, and Availability

Account lockout can slow guessing, but it can also be abused.

Lockout risk

An attacker may intentionally fail login attempts for the emergency department director and lock the account during a crisis.

Balanced design

Use rate limits, risk-based throttling, alerting, step-up authentication, and recovery paths that preserve clinical continuity without allowing easy bypass.

Recap: Password Policy

What to keep

Password policy should prevent weak choices at creation, limit online guessing, avoid unnecessary user burden, and support safe recovery. The goal is not theatrical complexity; it is resistance to realistic attacks.

In St. Isidore Hospital

A hospital password policy must work during a busy shift. If users cannot follow it, they will invent workarounds that weaken authentication.

Tokens and Smart Cards

Section Context: Tokens and Smart Cards

What we are talking about

Possession factors add something the user has to the authentication process. Tokens can be simple storage objects, cryptographic devices, smart cards, phones, or hardware security keys.

Hospital lens

A staff badge can unlock a door, but a smart card can also prove possession of a protected key during workstation or application login.

What Is a Token?

A token is a physical or logical object possessed by the user and used during authentication.

Common examples

Badge cards, magnetic stripe cards, memory cards, smart cards, hardware security keys, authenticator apps, and software certificates.

Hospital example

A staff badge can open doors. A smart card can also participate in computer login by proving possession of a cryptographic secret.

Card Types

Type	Authentication meaning
Embossed card	Carries visible identity information; weak as a digital authenticator.
Magnetic stripe	Stores simple data that can often be read or copied by compatible readers.
Memory card	Stores data electronically but does not process it securely.
Smart card	Includes processing capability and can execute authentication protocols.
Contact card	Communicates through exposed electrical contacts.
Contactless card	Communicates through a radio interface.

Memory Cards

Capability

Memory cards can store information but do not independently process secrets or run cryptographic protocols.

Security model

They are often combined with a PIN. The attacker needs both the card and the PIN, which is stronger than a password alone.

Hospital example

A badge plus PIN may unlock a medication cabinet. If the badge is stolen, the PIN still matters; if the PIN is observed, the badge still matters.

Memory Card Limitations

Reader dependency

Special readers increase cost and must themselves be protected.

Loss and replacement

Lost tokens interrupt work and require revocation, replacement, and sometimes identity re-verification.

User acceptance

A token used once a day may be accepted. A token required dozens of times per shift may create fatigue unless workflows are designed carefully.

Smart Cards

Smart cards are more capable than memory cards because they contain a microprocessor and memory.

Internal components

A smart card can include ROM for permanent data, EEPROM for applications and changing state, RAM for temporary data, I/O interfaces, and sometimes cryptographic acceleration.

Hospital example

A clinician smart card can store or protect a private key. During login, the card proves possession of that key without revealing it to the workstation.

Smart Card Authentication Protocols

Static

The card presents a fixed credential after the user unlocks it. This is simple but weaker if the credential can be copied.

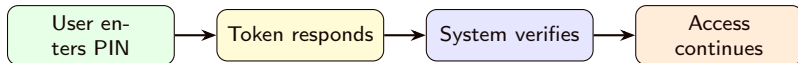
Dynamic password generator

The card generates changing codes. The verifier accepts a code only if it matches the expected value for that moment or counter.

Challenge-response

The system sends a fresh challenge. The card computes a response using a protected secret or private key.

Scheme: Token Plus PIN



Hospital example

For controlled-drug dispensing, the cabinet requires the nurse's badge and PIN before logging the withdrawal under her identity.

Recap: Tokens and Smart Cards

What to keep

Tokens strengthen authentication because attackers must compromise something the user possesses, not only something the user knows. Smart cards can protect secrets and participate in cryptographic protocols.

In St. Isidore Hospital

Badge workflows can improve security, but the hospital must manage readers, lost tokens, revocation, replacement, and user fatigue.

Remote Authentication

Section Context: Remote Authentication

What we are talking about

Remote authentication must cross a network, so the design must consider interception, replay, phishing, fake endpoints, and compromised home devices.

Hospital lens

When a clinician connects from home, the hospital is no longer protecting only a local workstation. It is protecting an authentication exchange across networks it does not fully control.

Local vs Remote Authentication

Local authentication

The user authenticates to a nearby system, such as a workstation, local terminal, or ATM-like device.

Remote authentication

Authentication data travels over a network. Attackers may observe traffic, replay old exchanges, or trick users into authenticating to a fake endpoint.

Hospital example

A doctor logging in from a ward workstation faces local session risks. A doctor connecting from home to the EHR also faces network interception, phishing, and endpoint compromise risks.

Replay Attacks

A replay attack reuses previously captured authentication data.

Problem

If the same password-derived response is valid every time, an attacker who records it once may use it later.

Hospital example

An attacker monitoring a poorly protected remote login captures a valid authentication exchange from Dr. Moretti and later sends the same response to the VPN gateway.

Challenge-Response Authentication

Challenge-response avoids sending a reusable secret.

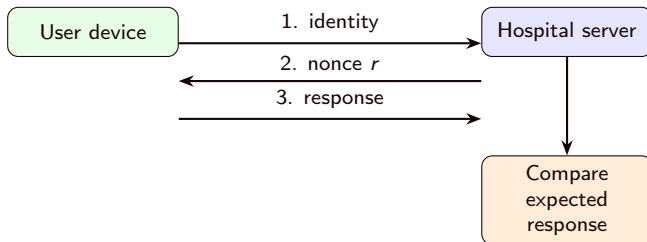
Algorithm

The server sends a fresh random value called a **nonce**. The client computes a response using the nonce and a secret-derived value. The server checks whether the response is correct for that nonce.

Security intuition

Because the nonce changes every time, yesterday's valid response should not authenticate today's login.

Scheme: Challenge-Response



What is protected

The password and its direct hash are not sent as reusable network messages.

Challenge-Response in the Hospital

Remote EHR access

The VPN gateway stores a verifier for Dr. Moretti. At login, it sends a nonce. Dr. Moretti's client computes a response using the nonce and the secret-derived value. A captured response is useless for the next nonce.

Important limit

Challenge-response protects against simple replay, but it does not automatically solve phishing, malware on the endpoint, stolen devices, weak recovery, or poor server validation.

Recap: Remote Authentication

What to keep

Remote authentication must avoid reusable secrets on the network. Nonces and challenge-response protocols make captured authentication exchanges less useful.

In St. Isidore Hospital

The hospital should treat remote access as higher risk than local access and require mechanisms that resist interception and replay.

Attacks Against Authentication

Section Context: Attacks Against Authentication



What we are talking about

Authentication can fail because a secret is guessed, a verifier database is stolen, a token is lost, a fake interface captures credentials, or a legitimate user is locked out.

Hospital lens

For a hospital, authentication attacks are not only privacy incidents. They can interrupt care, misattribute clinical actions, or allow unsafe changes to patient data.

Client-Side Attacks

Guessing

The attacker tries to authenticate without compromising the server or network. Rate limits, throttling, lockout design, and strong password policy reduce feasibility.

Token theft

If the system uses tokens, the attacker may need both the token and its activation factor, such as a PIN or biometric unlock.

Hospital example

A stolen badge should not be enough to open the EHR. The badge must be revoked quickly and should require an additional factor for sensitive systems.

Host-Side Attacks

Credential verifier theft

The attacker obtains password hashes, token seeds, passcodes, certificates, or biometric templates from the authentication server.

Defensive direction

Store only what is necessary, protect authenticators with strong access controls, use hardware security modules where appropriate, monitor privileged access, and avoid plaintext shared secrets.

Hospital example

If the MFA seed database for staff tokens is exposed, attackers may generate valid OTPs. The authentication server is therefore a high-value clinical security asset.

Interception and Observation

Physical observation

Shoulder surfing, written passwords, badge sharing, and unattended sessions can bypass otherwise strong controls.

Endpoint observation

Keyloggers, screen capture tools, malicious browser extensions, and compromised mobile devices can capture credentials or OTPs.

Hospital example

A shared workstation in a busy ward is a high-risk environment. Screen privacy, automatic lock, fast reauthentication, and user training all matter.

Trojan Horse Attacks

Idea

A fake application or device presents itself as legitimate and captures credentials.

Hospital example

A fake “EHR password reset” page sent by email asks Dr. Moretti for username, password, and OTP. If the attacker relays those values immediately, ordinary OTP MFA may still fail.

Mitigation

Use phishing-resistant authenticators when possible, train users on expected flows, validate domains, and monitor suspicious authentication patterns.

Denial of Service

Authentication can also be attacked by making legitimate login difficult or impossible.

System-wide DoS

The attacker floods the authentication service so staff cannot log in.

Targeted lockout

The attacker intentionally fails logins for a specific account until it is locked.

Hospital example

Locking the emergency physician's account during night shift may be as damaging as stealing it. Availability is part of authentication design.

Attack Summary

Attack	What fails	Typical defenses
Guessing	Weak secrets and unlimited attempts	Rate limits, blocklists, MFA
Host theft	Poorly protected verifier stores	Least privilege, slow hashes, HSMs, monitoring
Interception	Reusable credentials or weak endpoint	TLS, challenge-response, endpoint hardening
Replay	Static responses	Nonces, OTP, mutual authentication
Trojan horse	User trusts fake interface	Phishing-resistant MFA, user training, monitoring
DoS	Availability of login path	Throttling, resilient design, emergency access

Recap: Attacks Against Authentication

What to keep

Authentication attacks target people, endpoints, servers, networks, and recovery processes. A strong mechanism can still fail if the surrounding workflow is weak.

In St. Isidore Hospital

The hospital must protect both credential storage and everyday clinical authentication paths, especially shared workstations and remote access.

Multi-Factor and One-Time Passwords

What we are talking about

Multi-factor authentication reduces dependence on one credential.

One-time passwords are a common second factor, but they introduce secrets, time windows, enrollment, recovery, and phishing risks.

Hospital lens

The hospital may accept TOTP for ordinary remote login while requiring phishing-resistant authenticators for administrators and other high-impact accounts.

Two-Step Authentication

In many systems, users first enter a password and then provide a second value generated by a token or app.

Common pattern

First factor: username and password.

Second factor: one-time password, push approval, hardware token touch, smart-card response, or platform authenticator.

Hospital example

A clinician logs into the remote access portal with username and password, then enters a six-digit code from an authenticator app.

HOTP and TOTP

One-time passwords are usually generated from a shared secret and a changing value.

HOTP

HMAC-based OTP uses a counter. The token and server must agree on the counter value [M'Raihi et al., 2005].

TOTP

Time-based OTP replaces the counter with time steps. RFC 6238 defines TOTP as HOTP computed over a time value, commonly using 30-second windows [M'Raihi et al., 2011].

TOTP Enrollment

Shared secret

During enrollment, the server creates a unique secret for the user. The user's app stores the same secret, often transferred through a QR code.

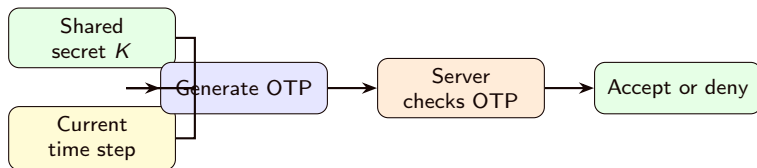
Synchronization

The app and server must share a compatible algorithm and a sufficiently synchronized notion of time. Network Time Protocol helps systems maintain accurate time.

Hospital example

Dr. Moretti scans a QR code into the authenticator app. From then on, the app and hospital server can generate matching codes for the same time window.

Scheme: TOTP Login



Validity window

Servers often accept a small time window around the current step to tolerate clock drift and network delay.

OTP Risks

Physical or logical token compromise

If the phone, app storage, or token seed is compromised, the attacker may generate valid codes.

Time-window attack

A captured OTP may be usable briefly. Real-time phishing can exploit this by relaying the code immediately.

Migration and recovery

Replacing a phone is operationally sensitive because secrets must be re-bound or migrated safely.

Same-Channel Problem

Two factors are stronger when they are independent. In practice, many factors meet on the same endpoint.

Problem

A user may type password and OTP into the same browser, with the same keyboard, on the same operating system. If that endpoint is compromised, both factors can be captured.

Hospital example

Malware on a home laptop can capture the VPN password and TOTP code during the same login. MFA still helps, but endpoint security remains essential.

Phishing-Resistant MFA

Some authenticators are designed to resist credential replay and phishing better than OTP.

Cryptographic authenticators

FIDO2/passkey-style authenticators use public-key cryptography and bind authentication to the legitimate service origin, making simple fake-site replay much harder [FIDO Alliance, 2026, National Institute of Standards and Technology, 2025].

Hospital example

For remote administrator access, the hospital may require a hardware security key. The key signs a challenge for the real hospital domain and does not reveal a reusable code to a fake page.

Recap: Multi-Factor and One-Time Passwords

What to keep

MFA improves authentication by combining factors, but OTP is not magic. Shared secrets, time synchronization, recovery, phishing, and endpoint compromise all remain design concerns.

In St. Isidore Hospital

The hospital can use OTP for many staff workflows, but should prefer phishing-resistant MFA for remote access, administrators, and high-impact clinical systems.

Authentication Lab

Section Context: Authentication Lab

What we are talking about

The lab turns the authentication mechanisms into small executable models. The goal is to see the logic of password verifiers, replay resistance, and TOTP MFA without pretending that a notebook is a production identity system.

Hospital lens

St. Isidore Hospital uses these examples to reason about policy choices. In real deployments, administrators configure identity providers, MFA platforms, operating-system authentication, logging, and recovery workflows.

Lab Setup

Environment

Use the lesson-local virtual environment: `lesson-05/.venv`

Notebooks

Open the notebooks in `lesson-05/examples` and select the `lesson-05/.venv` Python kernel.

Safety

The examples use artificial users and toy policies only. Do not use real patient data, real staff credentials, or production authentication secrets.

Notebook 1: Password Storage and Cracking

Open

```
lesson-05/examples/01_password_storage_and_cracking.ipynb
```

What it shows

The notebook compares plaintext password storage with salted PBKDF2 verifiers, then simulates a small offline dictionary attack against weak hospital-themed passwords.

Running example

Some St. Isidore accounts use predictable passwords such as hospital names, departments, or birth years. The attacker does not need the real login page once the verifier file is stolen.

Notebook 2: Challenge-Response and Replay

Open

```
lesson-05/examples/02_challenge_response_replay.ipynb
```

What it shows

The notebook first demonstrates a replayable static response, then adds a nonce so that each login attempt requires a fresh response.

Running example

A captured remote-login response from Dr. Moretti should not be reusable against the hospital VPN or EHR gateway during a later login attempt.

Notebook 3: TOTP Multi-Factor Authentication

Open

```
lesson-05/examples/03_totp_mfa.ipynb
```

What it shows

The notebook creates a TOTP shared secret, generates one-time codes, verifies current codes, and shows how verification windows handle clock drift.

Running example

Dr. Moretti's authenticator app and the hospital server can produce the same short code because they share a secret and a time step. The code expires, but real-time phishing is still possible.

Lab Recap



Takeaways

- Password hashes reduce damage only when combined with salts, slow hashing, and good policy.
- Challenge-response uses freshness to make captured responses less reusable.
- TOTP MFA improves security, but shared secrets, recovery, clock drift, phishing, and endpoint compromise still matter.

Connecting the Concepts

Section Context: Connecting the Concepts

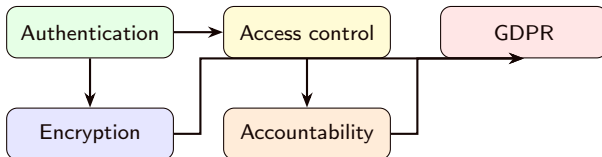
What we are talking about

Authentication is not an isolated feature. It supports access control, depends on encryption, feeds audit logs, and helps the organization demonstrate appropriate protection of health data.

Hospital lens

At St. Isidore Hospital, a login event becomes meaningful only when it is connected to permissions, protected sessions, recorded actions, and incident review.

Final Concept Map



Reading the map

Authentication names the actor. Access control limits the actor. Encryption protects authentication material and sessions. Logging makes actions accountable. GDPR gives the hospital a legal reason to make all of this reliable.

Final Recap

Authentication foundations

Identification presents an identity; verification proves the claim.
Authentication is not authorization, but authorization depends on it.

Mechanisms

Passwords, tokens, smart cards, biometrics, challenge-response, and OTP all provide evidence, but each has different failure modes.

Healthcare design lesson

Strong authentication must fit clinical reality: shared workstations, remote access, emergencies, usability pressure, audit needs, and sensitive patient data.

References I

[FIDO Alliance, 2026] FIDO Alliance (2026).

Fido2: Moving the world beyond passwords.

Available at <https://fidoalliance.org/fido2/>

.

[M'Raihi et al., 2005] M'Raihi, D., Bellare, M., Hoornaert, F., Naccache, D., and Ranen, O. (2005).

Hotp: An hmac-based one-time password algorithm.

RFC 4226.

Available at <https://www.rfc-editor.org/rfc/rfc4226>

.

[M'Raihi et al., 2011] M'Raihi, D., Machani, S., Pei, M., and Rydell, J. (2011).

Totp: Time-based one-time password algorithm.

RFC 6238.

Available at <https://www.rfc-editor.org/rfc/rfc6238>

.

References II

[National Institute of Standards and Technology, 2025] National Institute of Standards and Technology (2025).

Digital identity guidelines: Authentication and lifecycle management.

NIST Special Publication 800-63B, Revision 4.

Available at <https://pages.nist.gov/800-63-4/sp800-63b.html>

.

[Shirey, 2007] Shirey, R. (2007).

Internet security glossary, version 2.

RFC 4949.

Available at <https://www.rfc-editor.org/rfc/rfc4949>

.